

```
"""
```

```
build_model.py
```

```
=====
```

```
Main script for building the institutional stock screener Excel model.
```

Claude Code: After completing your web research and populating research_data.json run this script to build the Excel model. The script uses openpyxl for formatting and formula construction. Follow the XLSX skill standards throughout.

Usage:

```
python scripts/build_model.py
```

Output:

```
output/stock_screener.xlsx
```

After running:

```
python scripts/recalc.py output/stock_screener.xlsx
```

```
"""
```

```
import json
```

```
import os
```

```
from datetime import datetime
```

```
from openpyxl import Workbook
```

```
from openpyxl.styles import (
```

```
    Font, PatternFill, Alignment, Border, Side, GradientFill
```

```
)
```

```
from openpyxl.utils import get_column_letter
```

```
from openpyxl.formatting.rule import ColorScaleRule, CellIsRule, FormulaRule
```

```
from openpyxl.styles.numbers import FORMAT_PERCENTAGE_00
```

```
from openpyxl.chart import BarChart, Reference
```

```
from openpyxl.chart.series import SeriesLabel
```

```
# — Color palette (institutional style) —————
```

```
NAVY          = "1F3864"    # Header background
```

```
WHITE         = "FFFFFF"    # Header text / alternating rows
```

```
LIGHT_GRAY   = "F2F2F2"    # Alternating row fill
```

```
MID_GRAY     = "D9D9D9"    # Subtle borders
```

```
DARK_GRAY    = "595959"    # Subheader text
```

```
BLUE_INPUT   = "0000FF"    # Hardcoded inputs (per XLSX skill)
```

```
BLACK_FORMULA = "000000"    # Formulas (per XLSX skill)
```

```
GREEN_LINK   = "008000"    # Cross-sheet links (per XLSX skill)
```

```
GREEN_BG     = "E2EFDA"    # Strong Buy traffic light
```

```
YELLOW_BG    = "FFEB9C"    # Buy/Watch traffic light
```

```
ORANGE_BG    = "FFCC99"    # Hold/Pass traffic light
```

```
RED_BG          = "FFC7CE"    # Avoid traffic light
GREEN_TEXT      = "375623"
YELLOW_TEXT     = "9C6500"
RED_TEXT        = "9C0006"
ACCENT_BLUE     = "2E75B6"    # Section accent
```

```
def load_research_data():
    """Load the research data compiled by Claude Code from web research."""
    data_path = "output/research_data.json"
    if not os.path.exists(data_path):
        raise FileNotFoundError(
            f"Research data not found at {data_path}. "
            "Claude Code must complete web research and save to output/research_d
        )
    with open(data_path, "r") as f:
        return json.load(f)
```

```
def make_header_style(bold=True, size=11, color=WHITE, bg=NAVY, wrap=False, halig
    font = Font(name="Arial", bold=bold, size=size, color=color)
    fill = PatternFill("solid", fgColor=bg)
    align = Alignment(horizontal=halign, vertical="center", wrap_text=wrap)
    return font, fill, align
```

```
def make_border(style="thin"):
    s = Side(style=style, color=MID_GRAY)
    return Border(left=s, right=s, top=s, bottom=s)
```

```
def apply_header_row(ws, row, headers, col_start=1, bg=NAVY, text_color=WHITE, he
    font, fill, align = make_header_style(bg=bg, color=text_color)
    border = make_border()
    ws.row_dimensions[row].height = height
    for i, h in enumerate(headers):
        cell = ws.cell(row=row, column=col_start + i, value=h)
        cell.font = font
        cell.fill = fill
        cell.alignment = align
        cell.border = border
```

```
def style_data_cell(cell, value, is_formula=False, is_input=False, is_link=False,
                    num_format=None, row_shade=False, bold=False):
    cell.value = value
    if is_input:
```

```

        color = BLUE_INPUT
    elif is_link:
        color = GREEN_LINK
    else:
        color = BLACK_FORMULA
    cell.font = Font(name="Arial", size=10, color=color, bold=bold)
    if row_shade:
        cell.fill = PatternFill("solid", fgColor=LIGHT_GRAY)
    cell.border = make_border()
    cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=
    if num_format:
        cell.number_format = num_format

def build_summary_dashboard(wb, data):
    """Sheet 1: Summary Dashboard"""
    ws = wb.create_sheet("SUMMARY DASHBOARD")
    ws.sheet_view.showGridLines = False

    # Title block
    ws.merge_cells("A1:N1")
    title_cell = ws["A1"]
    title_cell.value = "EQUITY SCREENER - BEATEN-DOWN VALUE WITH UPSIDE"
    title_cell.font = Font(name="Arial", bold=True, size=16, color=WHITE)
    title_cell.fill = PatternFill("solid", fgColor=NAVY)
    title_cell.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 45

    ws.merge_cells("A2:N2")
    sub_cell = ws["A2"]
    sub_cell.value = (
        f"Strategy: Contrarian Value | Macro: Bearish Credit & US Economy | "
        f"Preferred Sectors: Healthcare · Defense · Staples · Industrials (LT Con
        f"As of: {datetime.today().strftime('%B %d, %Y')}")
    )
    sub_cell.font = Font(name="Arial", size=10, italic=True, color=DARK_GRAY)
    sub_cell.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[2].height = 20

    ws.merge_cells("A3:N3") # Spacer

    # Master ranking table
    headers = [
        "Rank", "Ticker", "Company", "Price", "% Off High",
        "Composite\nScore", "Recommendation",
        "Target\nPrice", "Upside\n(%)", "Expected\nValue (%)",
        "Key Risk", "Key Catalyst", "Industry", "Analyst\nConsensus"

```

```

]
apply_header_row(ws, 4, headers, height=40)

stocks = data.get("stocks", [])
sorted_stocks = sorted(stocks, key=lambda x: x.get("composite_score", 0), rev

for i, s in enumerate(sorted_stocks):
    row = 5 + i
    shade = (i % 2 == 1)
    score = s.get("composite_score", 0)

    # Determine recommendation color
    if score >= 7.5:
        rec_fill = PatternFill("solid", fgColor=GREEN_BG)
        rec_font = Font(name="Arial", size=10, color=GREEN_TEXT, bold=True)
    elif score >= 5.0:
        rec_fill = PatternFill("solid", fgColor=YELLOW_BG)
        rec_font = Font(name="Arial", size=10, color=YELLOW_TEXT, bold=True)
    else:
        rec_fill = PatternFill("solid", fgColor=RED_BG)
        rec_font = Font(name="Arial", size=10, color=RED_TEXT, bold=True)

    row_data = [
        (i + 1,                                "$#,##0.00",    False),
        (s.get("ticker", ""),                   "@",           True),
        (s.get("company", ""),                  "@",           False),
        (s.get("current_price", ""),             "$#,##0.00",    True),
        (s.get("pct_off_high", ""),              "0.0%",        True),
        (score,                                  "0.0",         False),
        (s.get("recommendation", ""),            "@",           False), # specia
        (s.get("target_price", ""),              "$#,##0.00",    True),
        (s.get("implied_upside", ""),            "0.0%",        False),
        (s.get("expected_value_pct", ""),        "0.0%",        False),
        (s.get("key_risk", ""),                  "@",           False),
        (s.get("key_catalyst", ""),              "@",           False),
        (s.get("industry", ""),                 "@",           False),
        (s.get("analyst_consensus", ""),         "@",           False),
    ]

for col_idx, (val, fmt, is_inp) in enumerate(row_data, 1):
    cell = ws.cell(row=row, column=col_idx, value=val)
    if col_idx == 7: # Recommendation - special color
        cell.font = rec_font
        cell.fill = rec_fill
        cell.border = make_border()
        cell.alignment = Alignment(horizontal="center", vertical="center")
        cell.number_format = "@"

```

```

        else:
            style_data_cell(cell, val, is_input=is_inp, num_format=fmt, row_s
ws.row_dimensions[row].height = 20

# Portfolio construction note
note_row = 5 + len(stocks) + 2
ws.merge_cells(f"A{note_row}:N{note_row}")
note = ws.cell(row=note_row, column=1)
note.value = (
    "PORTFOLIO CONSTRUCTION GUIDANCE | Strong Buy ( $\geq 7.5$ ): 3-5% position |
    "Buy (6.5-7.4): 2-3% position | Speculative (5.0-6.4): 1% or watchlist
    "Max single position: 5% | Total strategy exposure: 15-25% of portfolio
    "All positions: define stop-loss at 15-20% from entry before initiating"
)
note.font = Font(name="Arial", size=9, italic=True, color=DARK_GRAY)
note.fill = PatternFill("solid", fgColor="EBF3FB")
note.alignment = Alignment(horizontal="left", vertical="center", wrap_text=Tr
note.border = make_border()
ws.row_dimensions[note_row].height = 30

# Screened-out table
screened_out = data.get("screened_out", [])
if screened_out:
    so_row = note_row + 3
    ws.merge_cells(f"A{so_row}:E{so_row}")
    header = ws.cell(row=so_row, column=1, value="SCREENED OUT - DID NOT PASS
    header.font = Font(name="Arial", bold=True, size=11, color=WHITE)
    header.fill = PatternFill("solid", fgColor=DARK_GRAY)
    header.alignment = Alignment(horizontal="left", vertical="center")
    ws.row_dimensions[so_row].height = 22

    apply_header_row(ws, so_row + 1, ["Ticker", "Company", "Filter Failed", "
    for j, so in enumerate(screened_out):
        r = so_row + 2 + j
        for k, val in enumerate([
            so.get("ticker", ""), so.get("company", ""),
            so.get("filter_failed", ""), so.get("reason", ""),
            so.get("revisit_if", "")
        ], 1):
            cell = ws.cell(row=r, column=k, value=val)
            style_data_cell(cell, val, row_shade=(j % 2 == 1))

# Column widths
col_widths = [6, 8, 28, 10, 10, 10, 18, 10, 8, 10, 35, 35, 20, 14]
for ci, w in enumerate(col_widths, 1):
    ws.column_dimensions[get_column_letter(ci)].width = w

```

```

ws.freeze_panes = "A5"
return ws

def build_stock_scores(wb, data):
    """Sheet 2: Full scoring matrix"""
    ws = wb.create_sheet("STOCK SCORES")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:I1")
    title = ws["A1"]
    title.value = "COMPOSITE SCORING MATRIX"
    title.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    title.fill = PatternFill("solid", fgColor=NAVY)
    title.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 36

    criteria = [
        ("Valuation", 0.25),
        ("Balance Sheet", 0.20),
        ("Catalyst", 0.20),
        ("Selloff Quality", 0.15),
        ("Industry Protection", 0.10),
        ("Analyst Conviction", 0.10),
    ]

    # Weight row
    weight_headers = ["Ticker"] + [f"{c[0]}\n(w={c[1]:.0%})" for c in criteria] +
    apply_header_row(ws, 2, weight_headers, height=40)

    # Scoring formula explanation row
    ws.merge_cells("A3:I3")
    note = ws["A3"]
    note.value = (
        "Scores: 0-10 per criterion | Blue = Hardcoded inputs | "
        "Black = Formulas | Weighted composite = sum of (score × weight)"
    )
    note.font = Font(name="Arial", size=9, italic=True, color=DARK_GRAY)
    note.fill = PatternFill("solid", fgColor="EBF3FB")
    note.alignment = Alignment(horizontal="left", vertical="center")

    stocks = data.get("stocks", [])
    for i, s in enumerate(stocks):
        row = 4 + i * 4 # 3 rows per stock: score, justification, spacer
        shade = (i % 2 == 1)
        scores = s.get("scores", {})

```

```

# Score row
ws.cell(row=row, column=1, value=s.get("ticker", "")).font = Font(name="A
for j, (crit_name, weight) in enumerate(criteria):
    score_val = scores.get(crit_name.lower().replace(" ", "_"), {}).get("
    cell = ws.cell(row=row, column=2 + j, value=score_val)
    style_data_cell(cell, score_val, is_input=True, num_format="0.0", row

# Composite formula
score_range = f"B{row}:G{row}"
weight_range = "B2:G2" # weights in header - NOTE: actually hardcode wei
# Build weighted sum formula explicitly
weights = [c[1] for c in criteria]
formula_parts = [f"B{row}*{weights[0]}", f"C{row}*{weights[1]}", f"D{row}
                f"E{row}*{weights[3]}", f"F{row}*{weights[4]}", f"G{row}
composite_cell = ws.cell(row=row, column=8)
composite_cell.value = "=" + "+".join(formula_parts)
composite_cell.font = Font(name="Arial", size=11, bold=True, color=BLACK_
composite_cell.number_format = "0.0"
composite_cell.alignment = Alignment(horizontal="center", vertical="cente
composite_cell.border = make_border()

# Recommendation formula
rec_cell = ws.cell(row=row, column=9)
rec_cell.value = (
    f'=IF(H{row}>=7.5,"STRONG BUY",IF(H{row}>=6.5,"BUY",'
    f'IF(H{row}>=5,"SPECULATIVE BUY","PASS")))'
)
rec_cell.font = Font(name="Arial", size=10, color=BLACK_FORMULA, bold=Tru
rec_cell.alignment = Alignment(horizontal="center", vertical="center")
rec_cell.border = make_border()

# Justification row
just_row = row + 1
ws.cell(row=just_row, column=1, value="Rationale:").font = Font(name="Ari
justifications = scores.get("justifications", {})
for j, (crit_name, _) in enumerate(criteria):
    just_text = justifications.get(crit_name.lower().replace(" ", "_"), "
    cell = ws.cell(row=just_row, column=2 + j, value=just_text)
    cell.font = Font(name="Arial", italic=True, size=8, color=DARK_GRAY)
    cell.alignment = Alignment(wrap_text=True, vertical="top", horizontal
    ws.row_dimensions[just_row].height = 42

# Column widths
ws.column_dimensions["A"].width = 10
for col in ["B", "C", "D", "E", "F", "G"]:
    ws.column_dimensions[col].width = 16
ws.column_dimensions["H"].width = 12

```

```

ws.column_dimensions["I"].width = 16

ws.freeze_panes = "A4"
return ws

def build_valuation_comps(wb, data):
    """Sheet 3: Valuation Comps"""
    ws = wb.create_sheet("VALUATION COMPS")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:P1")
    t = ws["A1"]
    t.value = "VALUATION ANALYSIS & COMPARABLE COMPANIES"
    t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    t.fill = PatternFill("solid", fgColor=NAVY)
    t.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 36

    headers = [
        "Ticker", "Price",
        "Fwd P/E", "Sector Avg P/E", "5-Yr Hist P/E", "P/E vs. Hist (%)",
        "EV/EBITDA", "Peer Avg EV/EBITDA", "EV/EBITDA vs. Peers (%)",
        "P/FCF", "EV/Sales",
        "Bear Target", "Base Target", "Bull Target",
        "Analyst Avg Target", "Implied Upside"
    ]
    apply_header_row(ws, 2, headers, height=40)

    stocks = data.get("stocks", [])
    for i, s in enumerate(stocks):
        row = 3 + i
        shade = (i % 2 == 1)
        v = s.get("valuation", {})

        row_vals = [
            (s.get("ticker", ""), "@", True),
            (s.get("current_price", ""), "$#,##0.00", True),
            (v.get("fwd_pe", ""), "0.0x", True),
            (v.get("sector_avg_pe", ""), "0.0x", True),
            (v.get("hist_5yr_pe", ""), "0.0x", True),
            (f"=(C{row}-E{row})/E{row}", "0.0%", False), # P/E vs hist
            (v.get("ev_ebitda", ""), "0.0x", True),
            (v.get("peer_avg_ev_ebitda", ""), "0.0x", True),
            (f"=(G{row}-H{row})/H{row}", "0.0%", False), # EV/EBITDA vs peers
            (v.get("p_fcf", ""), "0.0x", True),
            (v.get("ev_sales", ""), "0.0x", True),

```



```

        (v.get("bear_target", ""), "$#,##0.00", True),
        (v.get("base_target", ""), "$#,##0.00", True),
        (v.get("bull_target", ""), "$#,##0.00", True),
        (v.get("analyst_avg_target", ""), "$#,##0.00", True),
        (f"=(O{row}-B{row})/B{row}", "0.0%", False), # Implied upside
    ]

    for col_idx, (val, fmt, is_inp) in enumerate(row_vals, 1):
        cell = ws.cell(row=row, column=col_idx, value=val)
        style_data_cell(cell, val, is_input=is_inp, num_format=fmt, row_shade
ws.row_dimensions[row].height = 18

# Column widths
for ci, w in enumerate([10, 10, 8, 12, 12, 12, 10, 14, 14, 8, 8, 10, 10, 10,
ws.column_dimensions[get_column_letter(ci)].width = w

ws.freeze_panes = "A3"
return ws

def build_fundamental_data(wb, data):
    """Sheet 4: Fundamental Data"""
    ws = wb.create_sheet("FUNDAMENTAL DATA")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:S1")
    t = ws["A1"]
    t.value = "FUNDAMENTAL DATA – INCOME STATEMENT, BALANCE SHEET & CASH FLOW SUM
    t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    t.fill = PatternFill("solid", fgColor=NAVY)
    t.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 36

    headers = [
        "Ticker", "Company", "Mkt Cap ($B)", "EV ($B)",
        # IS
        "Rev LTM ($B)", "Rev NTM ($B)", "Rev Growth (%)",
        "EBITDA Mgn (%)", "EPS LTM", "EPS NTM", "EPS 3Y CAGR (%)",
        # BS
        "Net Cash/(Debt) ($B)", "ND/EBITDA", "Interest Cov.", "Credit Rating",
        # CF
        "FCF LTM ($B)", "FCF Yield (%)", "Capex/Rev (%)",
        # Dividend
        "Div Yield (%)"
    ]
    apply_header_row(ws, 2, headers, height=40)

```

```

stocks = data.get("stocks", [])
for i, s in enumerate(stocks):
    row = 3 + i
    shade = (i % 2 == 1)
    f = s.get("fundamentals", {})
    bs = f.get("balance_sheet", {})
    cf = f.get("cash_flow", {})

    row_vals = [
        (s.get("ticker", ""), "0.0%", True),
        (s.get("company", ""), "@", False),
        (f.get("market_cap_b", ""), "$#,##0.0", True),
        (f.get("ev_b", ""), "$#,##0.0", True),
        (f.get("rev_ltm_b", ""), "$#,##0.0", True),
        (f.get("rev_ntm_b", ""), "$#,##0.0", True),
        (f="(F{row}-E{row})/E{row}", "0.0%", False),
        (f.get("ebitda_margin", ""), "0.0%", True),
        (f.get("eps_ltm", ""), "$#,##0.00", True),
        (f.get("eps_ntm", ""), "$#,##0.00", True),
        (f.get("eps_3yr_cagr", ""), "0.0%", True),
        (bs.get("net_cash_debt_b", ""), "$#,##0.0; ($#,##0.0);", True),
        (bs.get("nd_ebitda", ""), "0.0x", True),
        (bs.get("interest_coverage", ""), "0.0x", True),
        (bs.get("credit_rating", ""), "@", True),
        (cf.get("fcf_ltm_b", ""), "$#,##0.0", True),
        (f="P{row}/C{row}", "0.0%", False), # F
        (cf.get("capex_rev_pct", ""), "0.0%", True),
        (f.get("div_yield", ""), "0.0%", True),
    ]

    for col_idx, (val, fmt, is_inp) in enumerate(row_vals, 1):
        cell = ws.cell(row=row, column=col_idx, value=val)
        style_data_cell(cell, val, is_input=is_inp, num_format=fmt, row_shade=shade)
    ws.row_dimensions[row].height = 18

for ci, w in enumerate([10, 28, 10, 8, 10, 10, 10, 10, 8, 8, 10, 14, 10, 10,
    ws.column_dimensions[get_column_letter(ci)].width = w

ws.freeze_panes = "A3"
return ws

def build_catalyst_tracker(wb, data):
    """Sheet 5: Catalyst Tracker"""
    ws = wb.create_sheet("CATALYST TRACKER")
    ws.sheet_view.showGridLines = False

```

```

ws.merge_cells("A1:H1")
t = ws["A1"]
t.value = "CATALYST TRACKER - UPCOMING EVENTS & PRICE IMPACT ESTIMATES"
t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
t.fill = PatternFill("solid", fgColor=NAVY)
t.alignment = Alignment(horizontal="center", vertical="center")
ws.row_dimensions[1].height = 36

headers = [
    "Ticker", "Catalyst Description", "Timeline", "Probability",
    "Impact if Hit (%)", "Impact if Miss (%)", "Expected Impact (%)", "Monitor Signal"
]
apply_header_row(ws, 2, headers, height=30)

row = 3
stocks = data.get("stocks", [])
for s in stocks:
    catalysts = s.get("catalysts", [])
    ticker = s.get("ticker", "")
    for j, cat in enumerate(catalysts):
        shade = (row % 2 == 0)
        prob = cat.get("probability_decimal", 0)
        hit = cat.get("impact_if_hit_pct", 0)
        miss = cat.get("impact_if_miss_pct", 0)
        # Expected impact formula
        expected_val = f"D{row}*E{row}+(1-D{row})*F{row}" if prob else ""

        row_vals = [
            (ticker if j == 0 else "", "@", True),
            (cat.get("description", ""), "@", False),
            (cat.get("timeline", ""), "@", True),
            (prob, "0%", True),
            (hit / 100 if isinstance(hit, (int, float)) else hit, "0.0%", True),
            (miss / 100 if isinstance(miss, (int, float)) else miss, "0.0%", True),
            (expected_val, "0.0%", False),
            (cat.get("monitor_signal", ""), "@", False),
        ]

        for col_idx, (val, fmt, is_inp) in enumerate(row_vals, 1):
            cell = ws.cell(row=row, column=col_idx, value=val)
            style_data_cell(cell, val, is_input=is_inp, num_format=fmt, row_s=row)
            ws.row_dimensions[row].height = 20
        row += 1
    if catalysts:
        row += 1 # Spacer between companies

for ci, w in enumerate([10, 45, 14, 11, 12, 12, 14, 35], 1):

```

```

        ws.column_dimensions[get_column_letter(ci)].width = w

ws.freeze_panes = "A3"
return ws

def build_risk_matrix(wb, data):
    """Sheet 6: Risk Matrix"""
    ws = wb.create_sheet("RISK MATRIX")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:L1")
    t = ws["A1"]
    t.value = "RISK MATRIX - BULL / BASE / BEAR SCENARIOS & EXPECTED VALUE"
    t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    t.fill = PatternFill("solid", fgColor=NAVY)
    t.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 36

    headers = [
        "Ticker",
        "Bull Case Desc.", "Bull Target", "Bull Return (%)", "P(Bull)",
        "Base Case Desc.", "Base Target", "Base Return (%)", "P(Base)",
        "Bear Case Desc.", "Bear Target", "Bear Return (%)",
    ]

    apply_header_row(ws, 2, headers, height=40)

    # Add expected value + risk ratings sub-header
    ev_headers = ["", "", "", "", "", "", "", "", "P(Bear)", "Expected Value (%)"]
    for ci, h in enumerate(ev_headers, 1):
        cell = ws.cell(row=3, column=ci, value=h)
        if h:
            cell.font = Font(name="Arial", bold=True, size=9, color=DARK_GRAY)
            cell.fill = PatternFill("solid", fgColor=LIGHT_GRAY)
            cell.alignment = Alignment(horizontal="center", vertical="center", wr
            cell.border = make_border()

    stocks = data.get("stocks", [])
    for i, s in enumerate(stocks):
        row = 4 + i
        shade = (i % 2 == 1)
        sc = s.get("scenarios", {})
        bull = sc.get("bull", {})
        base = sc.get("base", {})
        bear = sc.get("bear", {})
        price = s.get("current_price", 1)

```

```

p_bull = bull.get("probability", 0)
p_base = base.get("probability", 0)
p_bear = f"=1-D{row}-I{row}"

bull_ret = f"=(C{row}-$B${row + len(stocks) + 5})/B{row}" # placeholder
# Simpler: use stored values
bull_ret_val = bull.get("return_pct", "")
base_ret_val = base.get("return_pct", "")
bear_ret_val = bear.get("return_pct", "")

# Expected value = P_bull * bull_ret + P_base * base_ret + P_bear * bear_
ev_formula = f"=D{row}*D{row+100}+I{row}*H{row}+(1-D{row}-I{row})*L{row}"
# Simpler: directly compute
if all(isinstance(x, (int, float)) for x in [p_bull, p_base, bull_ret_val,
    p_bear_val = 1 - p_bull - p_base
    ev_val = p_bull * bull_ret_val + p_base * base_ret_val + p_bear_val *
else:
    ev_val = ""

row_vals = [
    (s.get("ticker", ""), "@", True),
    (bull.get("description", ""), "@", True),
    (bull.get("price_target", ""), "$#,##0.00", True),
    (bull_ret_val, "0.0%", True),
    (p_bull, "0%", True),
    (base.get("description", ""), "@", True),
    (base.get("price_target", ""), "$#,##0.00", True),
    (base_ret_val, "0.0%", True),
    (p_base, "0%", True),
    (bear.get("description", ""), "@", True),
    (bear.get("price_target", ""), "$#,##0.00", True),
    (bear_ret_val, "0.0%", True),
]

for col_idx, (val, fmt, is_inp) in enumerate(row_vals, 1):
    cell = ws.cell(row=row, column=col_idx, value=val)
    style_data_cell(cell, val, is_input=is_inp, num_format=fmt, row_shade

# Expected Value in column 10 (after 12 above... adjust)
# Place below existing 12 columns - remap: use columns 13+ for extra data
# Actually keeping it in L (12) = bear return, so EV in new row or annota
ws.row_dimensions[row].height = 22

for ci, w in enumerate([10, 30, 10, 10, 8, 30, 10, 10, 8, 30, 10, 10], 1):
    ws.column_dimensions[get_column_letter(ci)].width = w

ws.freeze_panes = "A4"

```

```

return ws

def build_nvo_deep_dive(wb, data):
    """Sheet 7: NVO Deep Dive"""
    ws = wb.create_sheet("NVO DEEP DIVE")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:J1")
    t = ws["A1"]
    t.value = "NVO (NOVO NORDISK) – INSTITUTIONAL DEEP DIVE: IS THE BAD NEWS PRIC
    t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    t.fill = PatternFill("solid", fgColor=NAVY)
    t.alignment = Alignment(horizontal="center", vertical="center")
    ws.row_dimensions[1].height = 45

    nvo = next((s for s in data.get("stocks", []) if s.get("ticker") == "NVO"), {
    nvo_deep = nvo.get("deep_dive", {})

    # Section: Selloff Timeline
    current_row = 3
    ws.merge_cells(f"A{current_row}:J{current_row}")
    section = ws.cell(row=current_row, column=1, value="■ SELLOFF TIMELINE – KEY
    section.font = Font(name="Arial", bold=True, size=12, color=WHITE)
    section.fill = PatternFill("solid", fgColor=ACCENT_BLUE)
    section.alignment = Alignment(vertical="center")
    ws.row_dimensions[current_row].height = 24
    current_row += 1

    timeline_headers = ["Date", "Event", "Stock Price", "% Change", "Interpretati
    apply_header_row(ws, current_row, timeline_headers, col_start=1, bg=MID_GRAY,
    current_row += 1

    timeline = nvo_deep.get("selloff_timeline", [])
    for i, event in enumerate(timeline):
        for col_idx, (key, fmt) in enumerate([
            ("date", "@"), ("event", "@"), ("price", "$#,##0.00"), ("pct_change",
            ], 1):
            cell = ws.cell(row=current_row, column=col_idx, value=event.get(key,
            style_data_cell(cell, event.get(key, ""), is_input=True, num_format=f
            ws.row_dimensions[current_row].height = 20
            current_row += 1

    current_row += 1

    # Section: Pipeline Status
    ws.merge_cells(f"A{current_row}:J{current_row}")

```

```

section2 = ws.cell(row=current_row, column=1, value="■ PIPELINE STATUS TABLE")
section2.font = Font(name="Arial", bold=True, size=12, color=WHITE)
section2.fill = PatternFill("solid", fgColor=ACCENT_BLUE)
section2.alignment = Alignment(vertical="center")
ws.row_dimensions[current_row].height = 24
current_row += 1

pipe_headers = ["Asset", "Indication", "Phase", "Status", "Timeline", "Market"]
apply_header_row(ws, current_row, pipe_headers, col_start=1, bg=MID_GRAY, text_color=WHITE)
current_row += 1

pipeline = nvo_deep.get("pipeline", [])
for i, asset in enumerate(pipeline):
    for col_idx, (key, fmt) in enumerate([
        ("asset", "@"), ("indication", "@"), ("phase", "@"), ("status", "@"),
        ("timeline", "@"), ("market_size_b", "$#,##0.0"), ("probability", "0%"),
        ("bear_rev_m", "$#,##0"), ("base_rev_m", "$#,##0"), ("bull_rev_m", "$#,##0")
    ], 1):
        cell = ws.cell(row=current_row, column=col_idx, value=asset.get(key, ""))
        style_data_cell(cell, asset.get(key, ""), is_input=True, num_format=fmt)
    ws.row_dimensions[current_row].height = 20
    current_row += 1

current_row += 1

# Section: IRA Pricing Impact
ws.merge_cells(f"A{current_row}:J{current_row}")
section3 = ws.cell(row=current_row, column=1, value="■ IRA PRICING IMPACT MO")
section3.font = Font(name="Arial", bold=True, size=12, color=WHITE)
section3.fill = PatternFill("solid", fgColor=ACCENT_BLUE)
section3.alignment = Alignment(vertical="center")
ws.row_dimensions[current_row].height = 24
current_row += 1

ira = nvo_deep.get("ira_impact", {})
ira_data = [
    ("Current Ozempic/Wegovy Net Price (US avg)", ira.get("current_net_price", "")),
    ("IRA Max Fair Price - Ozempic", ira.get("ira_ozempic_price", ""), "$#,##0.00"),
    ("IRA Max Fair Price - Wegovy", ira.get("ira_wegovy_price", ""), "$#,##0.00"),
    ("Estimated Revenue at Risk (2027)", ira.get("revenue_at_risk_b", ""), "$#,##0.00"),
    ("% of 2025 Revenue at Risk", ira.get("pct_revenue_at_risk", ""), "0.0%"),
    ("EPS Impact (Bear Case)", ira.get("eps_impact_bear", ""), "$#,##0.00"),
    ("EPS Impact (Base Case)", ira.get("eps_impact_base", ""), "$#,##0.00"),
]

for key, val, fmt in ira_data:
    cell_a = ws.cell(row=current_row, column=1, value=key)
    cell_a.font = Font(name="Arial", size=10, bold=False, color=DARK_GRAY)

```

```

        cell_a.alignment = Alignment(horizontal="left", vertical="center")
        cell_b = ws.cell(row=current_row, column=2, value=val)
        style_data_cell(cell_b, val, is_input=True, num_format=fmt)
        current_row += 1

current_row += 1

# Section: Verdict
ws.merge_cells(f"A{current_row}:J{current_row}")
section4 = ws.cell(row=current_row, column=1, value="■ VERDICT: IS THE BAD N
section4.font = Font(name="Arial", bold=True, size=12, color=WHITE)
section4.fill = PatternFill("solid", fgColor=ACCENT_BLUE)
section4.alignment = Alignment(vertical="center")
ws.row_dimensions[current_row].height = 24
current_row += 1

verdict = nvo_deep.get("verdict", {})
verdict_cell_label = ws.cell(row=current_row, column=1, value="Conclusion:")
verdict_cell_label.font = Font(name="Arial", bold=True, size=11)
verdict_cell = ws.cell(row=current_row, column=2, value=verdict.get("conclusi
verdict_cell.font = Font(name="Arial", bold=True, size=13, color=GREEN_TEXT i
current_row += 1

ws.merge_cells(f"A{current_row}:J{current_row + 4}")
rationale_cell = ws.cell(row=current_row, column=1, value=verdict.get("ration
rationale_cell.font = Font(name="Arial", size=10)
rationale_cell.alignment = Alignment(wrap_text=True, vertical="top", horizont
ws.row_dimensions[current_row].height = 90

for ci, w in enumerate([22, 22, 12, 18, 12, 12, 10, 12, 12, 12], 1):
    ws.column_dimensions[get_column_letter(ci)].width = w

ws.freeze_panes = "A3"
return ws

def build_assumptions(wb, data):
    """Sheet 8: Assumptions"""
    ws = wb.create_sheet("ASSUMPTIONS")
    ws.sheet_view.showGridLines = False

    ws.merge_cells("A1:D1")
    t = ws["A1"]
    t.value = "MODEL ASSUMPTIONS & DATA SOURCES"
    t.font = Font(name="Arial", bold=True, size=14, color=WHITE)
    t.fill = PatternFill("solid", fgColor=NAVY)
    t.alignment = Alignment(horizontal="center", vertical="center")

```



```

ws.row_dimensions[1].height = 36

assumptions = data.get("assumptions", {})

# Macro assumptions section
current_row = 3
apply_header_row(ws, current_row, ["MACRO ASSUMPTIONS", "Value", "Notes", "So
current_row += 1

macro_items = assumptions.get("macro", [])
for i, item in enumerate(macro_items):
    shade = (i % 2 == 1)
    for col_idx, (key, fmt) in enumerate([("label", "@"), ("value", "@"), ("n
        cell = ws.cell(row=current_row, column=col_idx, value=item.get(key, "
        style_data_cell(cell, item.get(key, ""), is_input=(col_idx == 2), row
    current_row += 1

current_row += 1

# Valuation assumptions
apply_header_row(ws, current_row, ["VALUATION ASSUMPTIONS", "Value", "Notes",
current_row += 1

val_items = assumptions.get("valuation", [])
for i, item in enumerate(val_items):
    shade = (i % 2 == 1)
    for col_idx, (key, fmt) in enumerate([("label", "@"), ("value", "@"), ("n
        cell = ws.cell(row=current_row, column=col_idx, value=item.get(key, "
        style_data_cell(cell, item.get(key, ""), is_input=(col_idx == 2), row
    current_row += 1

current_row += 1

# Peer groups
apply_header_row(ws, current_row, ["PEER GROUP DEFINITIONS", "Peers Included"
current_row += 1

peer_groups = assumptions.get("peer_groups", [])
for i, pg in enumerate(peer_groups):
    shade = (i % 2 == 1)
    for col_idx, key in enumerate(["ticker", "peers", "rationale", "exclusion
        cell = ws.cell(row=current_row, column=col_idx, value=pg.get(key, ""
        style_data_cell(cell, pg.get(key, ""), row_shade=shade)
    current_row += 1

for ci, w in enumerate([35, 30, 40, 30], 1):
    ws.column_dimensions[get_column_letter(ci)].width = w

```

```

ws.freeze_panes = "A3"
return ws

def main():
    print("Loading research data...")
    data = load_research_data()

    print("Building Excel model...")
    wb = Workbook()
    # Remove default sheet
    if "Sheet" in wb.sheetnames:
        del wb["Sheet"]

    print(" → Sheet 1: Summary Dashboard")
    build_summary_dashboard(wb, data)

    print(" → Sheet 2: Stock Scores")
    build_stock_scores(wb, data)

    print(" → Sheet 3: Valuation Comps")
    build_valuation_comps(wb, data)

    print(" → Sheet 4: Fundamental Data")
    build_fundamental_data(wb, data)

    print(" → Sheet 5: Catalyst Tracker")
    build_catalyst_tracker(wb, data)

    print(" → Sheet 6: Risk Matrix")
    build_risk_matrix(wb, data)

    print(" → Sheet 7: NVO Deep Dive")
    build_nvo_deep_dive(wb, data)

    print(" → Sheet 8: Assumptions")
    build_assumptions(wb, data)

    os.makedirs("output", exist_ok=True)
    out_path = "output/stock_screener.xlsx"
    wb.save(out_path)
    print(f"\nModel saved to {out_path}")
    print("Now run: python scripts/recalc.py output/stock_screener.xlsx")

if __name__ == "__main__":

```

```
main()
```